PHASE 4 · ADVANCED

DAY 18

Type Hints & Static Analysis

Writing self-documenting, mypy-verified Python — Optional, Union, and Generics

2 hrs	3	20	4
Session Length	Concept Files	Practice Problems	Core Topics

30-DAY PYTHON PROGRAMMING COURSE

Session Overview

Goal: Learners write functions and classes with complete type hints, run `mypy` to statically verify them, and use Optional, Union, and Generic types to model real-world uncertainty (missing values, mixed-type inputs, reusable containers) without losing type safety.

Outcome: By the end of the session, students can read a mypy error report, fix the mismatch it points to, and write a fully type-hinted, generic container class from scratch.

Session Timeline

Time	Segment	Format
0:00 – 0:15	Why type hints? Live demo of an untyped bug	Discussion + live coding
0:15 – 0:40	Variable & function annotations, collection hints	Guided coding (File 1)
0:40 – 0:55	Running mypy from the terminal, reading its output	Live terminal demo
0:55 – 1:20	Optional and Union types, narrowing with if/isinstance	Guided coding (File 2)
1:20 – 1:25	Short break	—
1:25 – 1:50	TypeVar and Generics — building a type-safe Stack	Guided coding (File 3)
1:50 – 2:00	Practice problems + wrap-up + Day 19 preview	Independent practice

> **TIP**
>
> Instructor note: this is the first day mypy is run as a real terminal command rather than just discussed. Make sure mypy is installed (`pip install mypy --break-system-packages`) on every student machine BEFORE the session starts — checking this during class wastes coding time.

Prerequisites & Preparation

Requirements

Requirement	Detail
Python version	3.12 (confirmed via `python --version`)
Completed	Day 17 — Advanced OOP (ABCs, Protocols, dataclasses)
Package	mypy — install with `pip install mypy --break-system-packages`
Editor	VS Code with Pylance (shows inline type errors as you type)
Files needed	day18_01/02/03_*.py, day18_practise_problems_*.py

Instructor Pre-Session Checklist

[ ] Confirm mypy is installed on every machine: python -m mypy --version
[ ] Open Pylance's inline squiggly-underline error display in VS Code as a live demo
[ ] Have the three Day 18 concept files open in separate tabs, in order
[ ] Pre-load pnr_database / delivery_fees sample dicts so live typos don't stall the room
[ ] Skim the Common Errors table (next page) so mypy jargon doesn't catch you off guard

Step-by-Step Instruction Guide

STEP 1 — Why Type Hints? (0:00 – 0:15)

"Raise your hand if you've ever passed the wrong type into a function and only found out when it crashed three functions later. Today we fix that — we'll write down what type every variable and function is supposed to hold, and let a tool called mypy check our work before we even run the program."

```
# Untyped — works, but hides intent
def calculate_gst_no_hints(amount, rate):
    return amount * (rate / 100)

# Typed — intent is explicit
def calculate_gst(amount: float, rate: float) -> float:
    return amount * (rate / 100)

>>> calculate_gst(1000, 18)
180.0
```

WATCH OUT

Type hints are annotations only — Python itself never enforces them at runtime. Prove it live: reassign a str into an int-annotated variable and show the program keeps running anyway.

STEP 2 — Annotations & Collections (0:15 – 0:40)

"Type hints work on plain variables too, and — since Python 3.9 — on the built-in collections directly. No imports needed for list, dict, tuple, or set hints."

```
riders: list[str] = ["Riya Deshmukh", "Sanjay Verma"]
delivery_fees: dict[str, float] = {"Mumbai": 35.0, "Pune": 25.0}
order_record: tuple[int, str, float] = (10246, "Chennai", 899.0)

>>> delivery_fees
{'Mumbai': 35.0, 'Pune': 25.0}
```

Have students open **day18_01_type_hints_basics.py** and run it top to bottom, reading the Output comments as a self-check.

STEP 3 — Running mypy (0:40 – 0:55)

"Now let's let a tool catch our mistakes for us. mypy reads your file WITHOUT running it, and compares every function signature against how it's actually called."

```
$ python -m mypy day18_01_type_hints_basics.py --strict
day18_01_type_hints_basics.py:89: error: Incompatible types in
  assignment (expression has type "str", variable has type "int")
Found 1 error in 1 file (checked 1 source file)
```

macOS / Linux: `python3 -m mypy filename.py` | Windows: `python -m mypy filename.py`

Common mypy Errors Table

Error Message	Meaning	Fix
Incompatible types in assignment	Value assigned doesn't match the declared type	Fix the value, or correct the annotation
Argument 1 has incompatible type	A function was called with the wrong type	Cast the argument or fix the caller
Function is missing a type annotation	A def has no hints, and --strict requires them	Add parameter and return type hints
Item "None" has no attribute "X"	Called a method on an Optional value without checking	Add `if value is not None:` guard first

DONE WHEN

Done when every student has intentionally introduced ONE type error, run mypy, read the error line number, and fixed it themselves.

STEP 4 — Optional & Union Types (0:55 – 1:20)

"A lot of real bugs come from forgetting that a lookup can fail. Optional[str] is Python's way of saying 'this is a str, or it might be missing entirely' — and it forces you to handle the missing case."

```
def find_pnr_status(pnr: str, known: dict[str, str]) -> Optional[str]:
    return known.get(pnr)

def describe_status(status: Optional[str]) -> str:
    if status is None:
        return "No record found for this PNR."
    return f"Status: {status.title()}"

>>> describe_status(find_pnr_status("0000000000", pnr_database))
No record found for this PNR.
```

Show the modern syntax side by side: `Optional[str]` == `str | None`, and `Union[int, float]` == `int | float` (Python 3.10+).

> **WATCH OUT**
>
> Students commonly call a method on an Optional value without checking for None first — mypy will flag this as 'Item "None" has no attribute...'. Emphasise: always narrow (if x is None / isinstance check) before using the value.

STEP 5 — TypeVar & Generics (1:25 – 1:50)

"Last piece: what if a function or class needs to work with ANY type, but you still want mypy to track exactly which type it is? That's what TypeVar and Generic are for — think of them as a fill-in-the-blank for a type."

```
T = TypeVar("T")

class Stack(Generic[T]):
    def __init__(self) -> None:
        self._items: list[T] = []
    def push(self, item: T) -> None:
        self._items.append(item)
    def pop(self) -> T:
        return self._items.pop()

>>> task_stack: Stack[str] = Stack()
>>> task_stack.push("Renew Swiggy One")
>>> task_stack.pop()
'Renew Swiggy One'
```

Walk through why `list[Any]` would have silently allowed mixing types inside the same Stack, while `Stack[str]` keeps every push/pop consistent and mypy-checked.

DONE WHEN

Done when every student has built their own generic Stack or Queue class, instantiated it twice with two different type parameters (e.g. Stack[str] and Stack[int]), and shown both work correctly.

Session Wrap-Up

Student Checklist

[ ] I can annotate a variable and a function's parameters + return type
[ ] I can run mypy from the terminal and read its error output
[ ] I can use Optional[X] / X | None and narrow it with an `is None` check
[ ] I can use Union[X, Y] / X | Y and narrow it with isinstance()
[ ] I have built at least one generic class using Generic[T]

Key Concepts Recap

Concept	One-line Summary
Type hint	name: Type = value — informational, not enforced at runtime
mypy	Command-line static checker: python -m mypy file.py
Optional[X]	Shorthand for Union[X, None] — value might be missing
Union[X, Y] / X | Y	Value could be one of several types
TypeVar / Generic[T]	A placeholder type resolved per call/instance, keeping safety

Next Day Preview

Day 19 — Context Managers: the with protocol (__enter__ / __exit__), the contextlib module, writing custom context managers, and the @contextmanager decorator for clean resource handling.

Troubleshooting Notes

If mypy reports errors in files it wasn't asked to check (e.g. site-packages), students likely ran mypy on a folder instead of a single file — scope it down with the exact filename. If Pylance shows different errors than mypy, that's expected: Pylance is a faster, slightly different type checker built into VS Code, not a replacement for running mypy directly.

Quick Reference Card

Daily Workflow Commands

```
$ pip install mypy --break-system-packages
$ python day18_01_type_hints_basics.py
$ python -m mypy day18_01_type_hints_basics.py --strict
$ python day18_practise_problems_01_type_hints_basics.py
```

VS Code Shortcuts

Shortcut	Action
Ctrl/Cmd + Space	Trigger autocomplete (shows inferred types)
Hover over a variable	Pylance shows its inferred/declared type
Ctrl/Cmd + .	Quick-fix suggestions (e.g. add missing import)
F12	Go to definition of a type or function

Tool Summary

Tool / Symbol	Use
`list[X], dict[K, V], tuple[...], set[X]`	Built-in generic collection hints (3.9+)
`Optional[X]  /  X | None`	Value might be None
`Union[X, Y]  /  X | Y`	Value is one of several types
`TypeVar("T")`	Declares a placeholder type
`Generic[T]`	Base class for a generic (type-parameterised) class
`mypy --strict`	Runs the strictest static check available